

ALE 課程 3：IDoc 基礎結構與監控

Lecture

ale02 已經把自我迴圈管線搭好，並用 WE19 送出了一筆 MATMAS IDoc，讓它繞了一圈回到自己。這一題不會再建新的環境設定，而是把焦點放回那筆（或任何一筆）IDoc 本身：它內部到底長什麼樣子、怎麼用監控交易看懂它現在的狀態、怎麼用 WE19 的另一個模式（複製既有 IDoc）重新測試。

IDoc 的三層結構，逐欄拆解

ale01 講過 IDoc 分 Control / Data / Status 三種 Record，這裡把每一種背後對應的字典表與關鍵欄位攤開來看（這三張表都是標準 SAP 字典表，數十年沒變過，SE11 / SE16 都查得到）：

1. Control Record（表頭）——字典表 EDIDC，一筆 IDoc 只有一筆 Control Record

欄位	意義
DOCNUM	IDoc 號碼（系統流水號，這是識別一筆 IDoc 最核心的 Key）
DIRECT	方向：1 = Inbound（送進來）、2 = Outbound（送出去）
IDOCTYP	Basic Type（如 MATMAS05）
CIMTYP	Extension Type（如果有自訂擴充，見 ale06；沒有就是空的）
MESTYP	Message Type（如 MATMAS）
SNDPOR	發送方 Port
SNDPRN / SNDPRT	發送方 Partner No. / Partner Type（如 LS = Logical System）
RCVPOR	接收方 Port
RCVPRN / RCVPRT	接收方 Partner No. / Partner Type
CREDAT / CRETIM	建立日期 / 時間
STATUS	這筆 IDoc 目前最新的狀態碼——注意這是 Control Record 上的一個欄位，是「目前狀態」的快照，完整的狀態變化歷史要看 Status Record（見下方）

2. Data Record（實際資料）——字典表 EDID4，一筆 IDoc 有多筆 Data Record

欄位	意義
DOCNUM	對應哪一筆 IDoc（外鍵指回 EDIDC）
SEGNUM	這個 Segment 在整筆 IDoc 裡的序號（決定順序與層級）
SEGNAM	Segment 名稱（如 E1MARAM，對應 WE31 定義、本質是 DDIC Structure 的那個物件）

欄位	意義
PSGNUM	上層 (Parent) Segment 的序號——IDoc 的 Segment 可以有階層關係 (例如訂單表頭 Segment 底下掛多筆明細 Segment)，這個欄位記錄父子關係
SDATA	實際資料，以一整條固定長度的純文字字串存放 (1000 字元)，這個 Segment 定義的每個欄位依序、左靠齊、空白補滿存在這條字串裡——這就是 ale01 提到「IDoc Segment 全部欄位型別都是 <code>abap.char(n)</code> 」背後的原因：因為底層本來就是這樣一整條純文字，Segment Structure 只是告訴系統「這一串文字第幾個字元到第幾個字元代表哪個業務欄位」

3. Status Record (處理歷程) ——字典表 **EDIDS**，一筆 IDoc 有多筆 Status Record (每次狀態變化增加一筆，不會覆蓋舊的)

欄位	意義
DOCNUM	對應哪一筆 IDoc
COUNTER	第幾筆狀態記錄 (01、02、03.....依時間順序遞增)
STATUS	狀態碼
STATXT	狀態說明文字 (人類看得懂的那句話)
LOGDAT / LOGTIM	這個狀態發生的日期/時間

這代表 **EDIDC-STATUS** 跟 **EDIDS** 裡最後一筆的 **STATUS** 應該永遠一致——**EDIDC** 只保留「目前」狀態方便快速查詢，**EDIDS** 才是完整的處理歷程，除錯時要看 **EDIDS** 才能知道「它是怎麼一步步走到現在這個狀態的」，不能只看 **EDIDC**。

常見狀態碼——這門課會反覆用到的一小張表

狀態碼很多 (官方定義超過 70 種)，訓練用途只要記住下面這幾個，涵蓋自我迴圈情境會遇到的所有正常/異常路徑：

狀態碼	方向	意義
03	Outbound	Data passed to port OK——IDoc 已交給通訊層 (tRFC)，送出這一步本身沒有問題
12	Outbound	Dispatch OK——確認對方已經收到
29	Outbound	Error in ALE service——組 IDoc 或送出過程本身出錯 (例如 Partner Profile 設定有誤)
64	Inbound	IDoc ready to be transferred to application——已經收到，正準備交給 Inbound Function Module 處理
53	Inbound	Application document posted ——完整成功，應用層資料已經真的寫進去了 (自我迴圈測試的理想終點)
51	Inbound	Application document not posted (error) ——收到了，但 Inbound Function Module 執行失敗 (例如資料不合法，或 ale08 會教的自訂驗證邏輯擋下來)

狀態碼	方向	意義
56	Inbound	IDoc with errors added——語法/結構層級就有問題，甚至沒能正常送進 Inbound 處理邏輯
68	Inbound	Error - no further processing——找不到對應的 Process Code 或處理邏輯中斷 (ale02 常見誤區 Q&A 提過的「Process Code 填錯」就會停在這裡附近)

WE02 vs. WE05——兩個監控交易的差異

- **WE02 (IDoc List)**：基本款，選取畫面可以用 Message Type / Basic Type / 狀態 / 日期範圍等條件篩選，結果是一個可以展開的樹狀清單 (Control Record → Data Record → Status Record 逐層展開)，適合「我知道大概要找哪幾筆，想細看內容」。
- **WE05 (IDoc Lists，擴充版)**：篩選條件更多 (可以按 Partner、Direction、更細的日期時間範圍)，結果預設是清單型 (一列一筆 IDoc，欄位化顯示狀態/方向/日期)，適合「我想看一段時間內大量 IDoc 的整體狀況/統計」，比較像是監控儀表板的角色。兩者背後查的是同一批資料 (**EDIDC / EDID4 / EDIDS**)，差異在畫面設計跟篩選條件的豐富度，不是兩套不同的資料。

WE19 (Test Tool) 的第二種模式：複製既有 IDoc

ale02 用的是 **WE19** 的 **Existing Data** 模式 (拿一筆現有的應用資料，如物料主檔，讓系統幫你組一筆全新的 IDoc)。這題要用另一種模式——**Existing IDoc**：直接輸入一個已經存在的 IDoc 號碼，把它整份複製一份到編輯畫面，可以在畫面上直接修改任何一個 Segment 裡的任何一個欄位值，改完之後可以選擇：

- **Standard Outbound Processing**：把改過的內容當成一筆新的 Outbound IDoc 重新送出 (模擬「如果原始資料是這個值，會怎麼送」)
- **Inbound Processing**：把改過的內容直接當成一筆 Inbound IDoc 送進 Inbound 處理邏輯 (**這是實務上更常用的模式**——不需要真的有人送一筆新 IDoc 過來，就能重複測試 Inbound Function Module / 自訂驗證邏輯對各種資料內容的反應，ale08 會大量用到這個技巧來測試自訂 Inbound 客製化)

學習目標

- 能講出 Control / Data / Status 三種 Record 分別對應哪張字典表、各自的關鍵欄位是什麼
- 理解 **EDIDC-STATUS** (快照) 跟 **EDIDS** (完整歷程) 的關係，除錯時知道要查哪一個
- 認得本題列出的常見狀態碼，看到狀態碼能講出下一步該往哪個方向排查
- 能分辨 **WE02 / WE05** 的使用情境差異
- 能用 **WE19** 的「Existing IDoc」模式複製一筆 IDoc、修改欄位、重新送出 (Outbound 或 Inbound 皆可)

事前準備

延續 ale02 建好的自我迴圈環境 (**SM59 / WE21 / WE20 / BD64** 皆已完成)，且已經用 **WE19** 送出至少一筆 **MATMAS** IDoc。如果 ale02 尚未完成或那筆測試 IDoc 找不到了，可以直接在本題步驟 1 用 **WE02** 重新搜尋確認，或回頭用 ale02 步驟 7 的方法再送一筆新的。

題目需求

步驟 1：用 **WE02** 找到 ale02 送出的 IDoc

1. 交易碼 **WE02**
2. 選取畫面填：**Message Type** = **MATMAS** · **Direction** 留空（先看全部方向）· **Created On** 填今天或 ale02 操作那天的日期
3. Execute · 找到你的那筆 IDoc（如果只有一筆最簡單；如果有多筆，找 **Receiver** 是 **S4HCLNT130** 的那一筆）
4. 雙擊展開樹狀結構，依序點開 **Control Record**（核對 **Direction** / **Sender** / **Receiver** / **Message Type** 是否跟預期一致）、**Data Records**（找到 **E1MARAM** 這個 Segment，看看實際資料內容）、**Status Record**（看完整的狀態變化清單，注意 **Counter** 是不是依序遞增，每一筆的狀態說明文字在講什麼）

步驟 2：用 **WE05** 看整體狀況

1. 交易碼 **WE05**
2. 選取畫面條件放寬一點（例如只填 **Direction** 或日期範圍，不填 **Message Type**）· Execute
3. 觀察清單畫面：這個 Client 目前總共有多少筆 IDoc？各種狀態各有幾筆？（如果數量夠多，畫面上通常有依狀態分類的統計）
4. 對照步驟 1 找到的那一筆，確認在這個清單型畫面上看到的資訊跟 **WE02** 展開的內容是否一致（驗證「兩者查的是同一批資料，只是呈現方式不同」這個結論）

步驟 3：用 **WE19** 複製既有 IDoc 並修改重送（Inbound Processing 模式）

1. 交易碼 **WE19**
2. 切到畫面最上方，直接在初始畫面填入步驟 1 找到的 IDoc 號碼（**Existing IDoc** 欄位）· Execute——系統會把這筆 IDoc 完整複製到編輯畫面
3. 在左側樹狀結構點開 **E1MARAM** Segment，找到一個文字類欄位（例如物料的某個描述性欄位），把值改成為一個明顯不同、容易辨認的測試值（例如加上 **_TEST03** 後綴）
4. 工具列選 **Inbound Processing**（不是 Standard Outbound Processing——這次要測的是「如果 Inbound 收到這個修改過的內容會怎麼處理」）
5. 系統會產生一筆全新的 **IDoc 號碼**（跟原本複製來源的號碼不同），並直接跑一次 Inbound 處理

步驟 4：回頭用 **WE02** 確認新 IDoc 的結果

1. 交易碼 **WE02**，用步驟 3 產生的新 IDoc 號碼查詢
2. 展開 **Status Record**，確認最終狀態碼是什麼——理想情況下應該會跟 ale02 那筆一樣走到 **53**（因為改的只是一個描述性欄位，不影響 Key 欄位的合法性）
3. 如果卡在 **51** / **56** / **68**，展開 Status Record 看 **STATXT** 的錯誤說明文字是什麼，記下來回報，我可以用 [datapreview/freestyle](#) 查 **EDIDS** 幫你確認完整訊息內容並排查

參考答案（驗證方式）

操作完成後回報：① 步驟 1 找到的原始 IDoc 號碼與其 Status Record 完整歷程 ② 步驟 3/4 產生的新 IDoc 號碼與最終狀態碼。我會用以下查詢幫你交叉確認（表名/欄位皆為標準 SAP 字典表，可直接查）：

- `SELECT DOCNUM, DIRECT, MESTYP, STATUS, SNDRN, RCVPRN FROM EDIDC WHERE DOCNUM IN ('<原始IDoc號>', '<新IDoc號>')`
- `SELECT DOCNUM, COUNTER, STATUS, STATXT FROM EDIDS WHERE DOCNUM = '<新IDoc號>' ORDER BY COUNTER`
- `SELECT DOCNUM, SEGNUM, SEGNAM, SDATA FROM EDID4 WHERE DOCNUM = '<新IDoc號>' AND SEGNAM = 'E1MARAM'`——確認你改的那個欄位值真的反映在 **SDATA** 這條純文字字串裡對應的位置

思考題

1. **EDIDC-STATUS** 跟 **EDIDS** 裡最新一筆的 **STATUS** 理論上應該一致，但如果哪天你發現兩者不一致，可能代表什麼問題？（提示：想想這兩個欄位分別是「誰」、在「什麼時候」寫入的）
2. **WE19** 的「Inbound Processing」模式讓你不需要真的有一筆從外部送進來的 IDoc，就能測試 Inbound 邏輯。這對開發階段有什麼好處？但如果你完全依賴這個工具測試、從來沒有測過真正的 Outbound→Inbound 端對端流程，可能會漏掉哪一類問題？（提示：想想 Port / Partner Profile / 通訊層這些東西，**WE19** Inbound Processing 模式有沒有真的經過它們）
3. **EDID4** 的 **SDATA** 把所有欄位擠進一條固定長度的純文字字串，如果某個 Segment 未來要新增一個欄位，這對既有、已經在流通的舊版 IDoc 資料會有什麼影響？（提示：這也是為什麼 IDoc Basic Type 會有版本號，如 **MATMAS03** / **MATMAS05**，而不是直接改同一個版本）

答案

本題全程為 SAP GUI 操作與監控查詢，沒有可快照的 ABAP 原始碼。完成後請回報步驟 1 / 3 / 4 的 IDoc 號碼與狀態碼，我會用 `datapreview/freestyle` 查 **EDIDC** / **EDIDS** / **EDID4** 協助確認結果。